INDIVIDUAL PROJECT · PROGRAMMING & METHODS · BERGHS

Dailies

How an AI API writes the summaries on my blog

Elias Grünewald Dykert

elias-blog-xi.vercel.app · github.com/elijaho-keh/elias-blog



Röktanke · AI image · 2026

THE BLOG

A behind-the-scenes blog about my own work

THE NAME

In film, dailies are the raw footage you watch at the end of each shooting day, before the edit.

THE PURPOSE

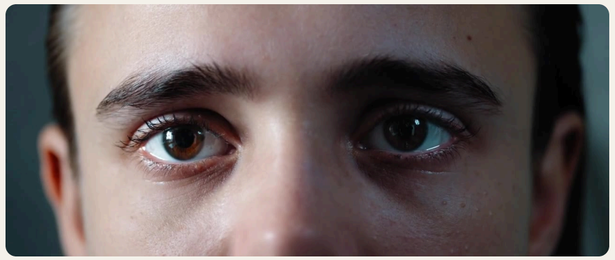
The process, ideas and failures behind my film, AI and art projects.

THE AUDIENCE

Classmates, teachers, creative agencies and people who find my portfolio.

THE CONTENT

Eight posts (Film 4, AI 3, Art 1), written from eliasdykert.se.







DEMO · 2 MINUTES

What I’ll show on the live site

1

Home

Search “memory”, filter by Film, try a nonsense word

2

A post

AI summary, video, Kindred work, leave a comment

3

Theme

Switch between dark and light

4

Write

Log in, publish a short post, watch the summary appear

Plain HTML, CSS and JavaScript · Vercel serverless functions · PostgreSQL on Neon · Gemini AI · Cleveland Museum of Art API

1 · THE CONCEPT

An AI API lets your own code ask an AI model

WHAT IT IS

Like ChatGPT without the chat window. My code sends text over the internet and gets text back.

WHY IT'S USEFUL

The AI does a writing task inside my own site, automatically, every time I publish.

WHAT YOU'LL LEARN

What you send, what comes back, how I built it, and what went wrong.

API = application programming interface: a way for one program to ask another program for something.

1 · THE CONCEPT

My use: a two-sentence summary on every post



Flora: a piano trio on an empty stage

Film · 346 words of text

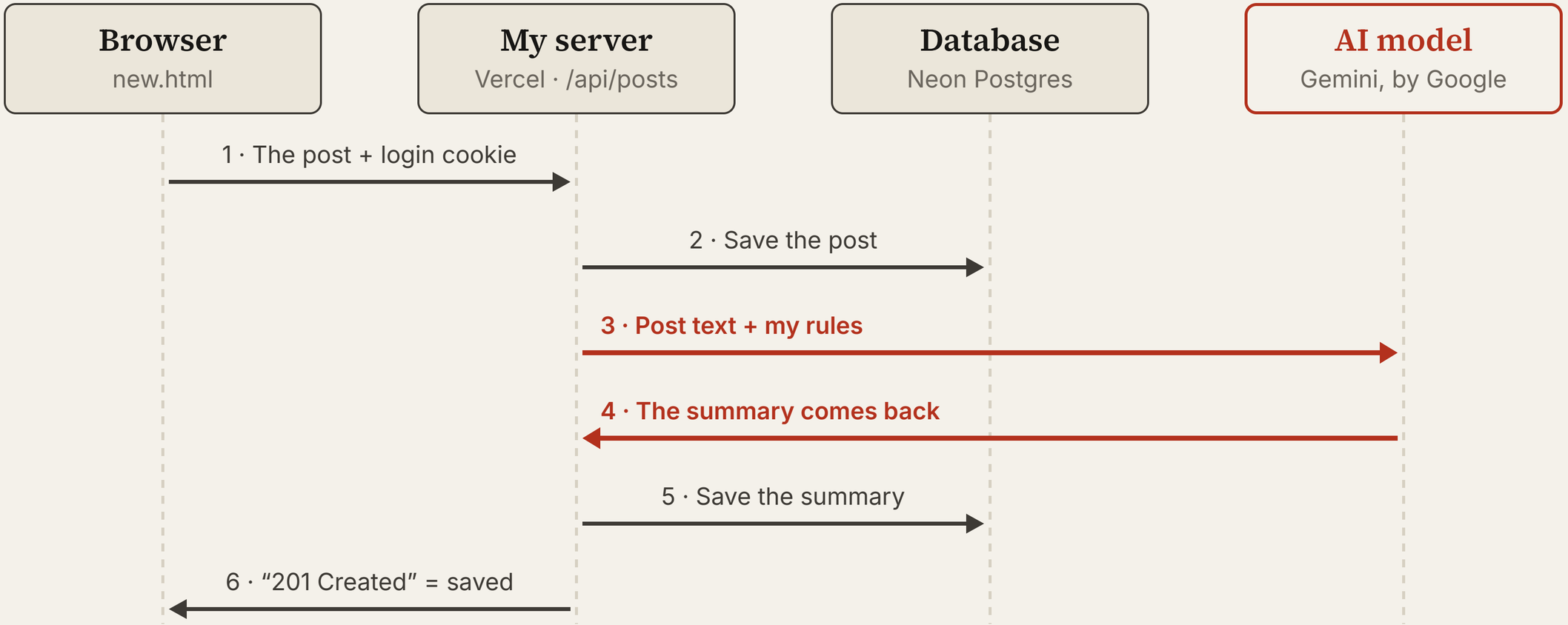
Gemini

AI SUMMARY

Ebba Dankel performs with her trio on an empty concert hall stage for her new music video. The minimalist filming style highlights the physical effort of the music and the connections between the musicians.

Made when the post is published · saved in the database · shown on the home page and under the post title

What happens when I click Publish



Six words you'll hear

Word	What it means	On my blog
API key	A secret password that proves the request is mine	GEMINI_API_KEY, only on the server
Endpoint	The web address the request is sent to	…/openai/chat/completions
Model	Which AI writes the answer	gemini-3.5-flash-lite
Prompt	What you send: rules (system) + content (user)	My rules + the post text
Token	A piece of text, about ¾ of a word	max_tokens: 160
Temperature	How random the answer is: 0 = always the same	0.3: steady, not creative

What my server sends, and what comes back

The request · sent with POST, the API key in a header

```
{
  "model": "gemini-3.5-flash-lite",
  "max_tokens": 160,
  "temperature": 0.3,
  "messages": [
    { "role": "system", "content": "You write the short summaries…" },
    { "role": "user", "content": "Flora is the first music video…" }
  ]
}
```

The answer · shortened

```
{
  "choices": [{
    "message": {
      "role": "assistant",
      "content":
       "Ebba Dankel performs…"
    }
  }]
}
```

My code needs one value from the answer: data.choices[0].message.content

Both sides use JSON (JavaScript Object Notation), a text format for data.

3 · MY IMPLEMENTATION

Save first, ask the AI second

api/posts.js · shortened

```
// 1. Who is asking? 401 if not logged in
const user = await requireUser(req, res);
if (!user) return;
// 2. … check every field …

// 3. Save the post BEFORE asking the AI
const post = await createPost(values, user.id);

// 4. Ask the AI (null if it fails)
const summary = await trySummarize(post.body);
if (summary) {
  await updateSummary(post.id, summary);
}
```

The AI can fail

It can be slow, down or out of free requests. My code waits at most 8 seconds.

Failing is fine

trySummarize() catches the error and gives back null. The post is already saved.

The key stays hidden

Only this server code has the key. The browser never talks to Gemini.

3 · MY IMPLEMENTATION

The prompt is a list of rules, and the model still breaks them

MY RULES · THE SYSTEM PROMPT

- Exactly two sentences, at most 45 words
- Present tense, neutral voice
- Never “I”, “this post” or “the text”
- Only use facts that are in the post
- Reply with the two sentences only

What happened	What I did
Began with “This post showcases…”	Made the rule stricter, read every summary
Invented “astronauts” and a “choir”	Re-ran them; labelled “AI summary” on the page
Gibberish test post: answer too short	My code rejected it; saved without a summary

Choosing a model: three tested, one chosen

Model	What happened
gemini-2.5-flash-lite	Error 404: “no longer available to new users”
gemini-3.5-flash	Empty answer: a “thinking” model that spent all 160 tokens thinking
gemini-3.5-flash-lite	**Chosen:** two clean sentences in 0.6–1.3 seconds, free
claude-haiku-5.5 via OpenRouter	My first plan, never tested: OpenRouter’s site was down. Still in the code as an option.

What mattered: free, fast, follows the rules, fits in 160 tokens.

3 · MY IMPLEMENTATION

How I worked: docs, AI help and dead ends

DOCUMENTATION

- The assignment brief
- Google’s Gemini API docs: keys, models, free-tier limits
- OpenRouter’s docs and price list

PROMPTING AI

- Claude Code as a pair programmer
- One brief with all eight requirements, e.g. “If the AI call fails, the post still saves”
- One phase and one commit at a time, decisions logged in PROCESS.md

DEAD ENDS

- OpenRouter was down, so I switched to Gemini: same request format
- Error 429 at 15 requests a minute: the script logged it and carried on
- A fake key first: 401, and the post still saved

4 · RECAP

Four things to remember

1. An AI API: your code sends text and rules, and gets text back.
2. The key stays on the server. The browser never talks to the AI.
3. Save first, ask the AI second, so a failing AI can’t lose your data.
4. Check what comes back, and label it. AI is helpful, not the truth.

Questions?